

GAMEHUB MANAGER

Sistema de Gerenciamento para Lan House

DOCUMENTAÇÃO TÉCNICA DO SOFTWARE

Autores:

José Vitor Araújo Soares

Victor Daniel Pereira Nascimento

Projeto Final - Sistema Orientado a Objetos com Banco de Dados

Linguagem Java | Java Swing | JDBC | MySQL

2026

SUMÁRIO

- 1 INTRODUÇÃO
- 2 FUNDAMENTAÇÃO TEÓRICA
- 3 APRESENTAÇÃO DO SISTEMA
- 4 LEVANTAMENTO DE REQUISITOS
- 5 CASOS DE USO
- 6 DIAGRAMA DE ATIVIDADES
- 7 DIAGRAMA DE MÁQUINA DE ESTADOS
- 8 DIAGRAMAS DE SEQUÊNCIA
- 9 MODELO CONCEITUAL
- 10 MODELO RELACIONAL
- 11 DIAGRAMA DE CLASSES
- 12 ARQUITETURA DO SISTEMA
- 13 DESCRIÇÃO DAS INTERFACES
- 14 OBSERVAÇÕES TÉCNICAS DO CÓDIGO-FONTE
- 15 CONCLUSÃO
- 16 REFERÊNCIAS

1 INTRODUÇÃO

O GameHub Manager é um sistema desktop desenvolvido para apoiar a gestão operacional de uma lan house. O código-fonte recebido apresenta uma aplicação Java construída com interface gráfica Java Swing, persistência em banco de dados MySQL por meio de JDBC e organização em pacotes que separam telas, entidades de domínio, objetos de acesso a dados e classe utilitária de conexão.

A proposta do sistema é centralizar operações administrativas básicas: autenticação do administrador, cadastro e manutenção de clientes, cadastro e manutenção de funcionários, controle de computadores e registro de aluguéis. Essas funcionalidades aparecem diretamente nas telas JFrame e nas classes DAO presentes no projeto.

Esta documentação foi construída a partir da análise do código-fonte do projeto GameHub Manager e das normas do Projeto Final. O texto evita funcionalidades não implementadas e registra também limitações encontradas no estado atual do código, como autenticação fixa em tela e ausência de criptografia de senha no trecho implementado.

O documento apresenta requisitos, casos de uso, diagramas UML, modelo conceitual, modelo relacional, arquitetura e descrição das interfaces. O objetivo é servir como documentação técnica de entrega e também como referência para manutenção futura do sistema.

1.1 Objetivo Geral

Documentar tecnicamente o GameHub Manager, descrevendo as funcionalidades implementadas, a estrutura do código-fonte, os requisitos identificados, os modelos UML, o banco de dados inferido e as interfaces de usuário do sistema.

1.2 Objetivos Específicos

- Identificar os requisitos funcionais e não funcionais a partir das classes e telas existentes.
- Especificar os casos de uso do administrador da lan house.
- Representar os fluxos principais por diagramas de atividades, máquina de estados e sequência.
- Apresentar os modelos conceitual e relacional coerentes com os DAOs e classes de modelo.
- Descrever a arquitetura em camadas View, Model, DAO e Util.
- Registrar observações técnicas relevantes, inclusive divergências entre requisitos do trabalho e código implementado.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Engenharia de Software e documentação

A documentação técnica de software organiza as decisões de análise, projeto e implementação de um sistema. Em projetos acadêmicos, ela permite demonstrar que as funcionalidades implementadas possuem relação com requisitos, casos de uso, modelos de dados, estrutura de classes e comportamento das interfaces. No GameHub Manager, a documentação foi elaborada por engenharia reversa do código-fonte, isto é, partindo das classes Java existentes e das operações efetivamente programadas.

2.2 Programação Orientada a Objetos

A Programação Orientada a Objetos organiza o sistema em classes que representam conceitos do domínio. No projeto, as classes Cliente, Funcionario, Computador e Aluguel são objetos simples de domínio, contendo atributos privados, construtores e métodos de acesso. Essa estrutura segue o princípio de encapsulamento, pois os atributos são manipulados por métodos getters e setters em vez de serem acessados diretamente por outras partes da aplicação.

2.3 Java Swing

Java Swing é a biblioteca utilizada para criação das interfaces gráficas desktop. O sistema possui telas JFrame para login, menu principal e módulos de cadastro. Cada tela possui componentes como JTextField, JButton, JTable, JComboBox e JOptionPane. O tratamento das ações ocorre por métodos actionPerformed associados aos botões Insert, Update, Delete, Total e navegação.

2.4 JDBC e MySQL

O JDBC permite que a aplicação Java execute comandos SQL em um banco relacional. No projeto, os DAOs usam Connection, PreparedStatement e ResultSet para inserir, atualizar, excluir, buscar, listar e contar registros. As informações de conexão são carregadas de um arquivo database.properties e utilizadas pela classe ConnectionFactory para abrir conexão com o banco lanhousedb em MySQL.

2.5 Padrão DAO

O padrão DAO, Data Access Object, separa as regras de persistência do restante do sistema. No GameHub Manager, cada entidade principal possui um DAO correspondente: ClienteDAO, FuncionarioDAO, ComputadorDAO e AluguelDAO. A View cria objetos de modelo e chama métodos do DAO, enquanto o DAO encapsula os comandos SQL. Essa divisão reduz o acoplamento entre interface gráfica e banco de dados.

2.6 UML

A UML é utilizada para representar o sistema por diferentes perspectivas: casos de uso mostram funcionalidades do ponto de vista do ator; atividades descrevem fluxos de processo; máquina de estados descreve mudanças de estado de uma entidade; sequência mostra troca de mensagens entre objetos; classes representam atributos, métodos e dependências; modelo conceitual e relacional mostram o domínio e a organização dos dados.

3 APRESENTAÇÃO DO SISTEMA

O GameHub Manager é voltado ao controle administrativo de uma lan house. A aplicação é iniciada por uma tela de login chamada LoginJFrame. Quando o usuário informa as credenciais esperadas pelo código, a tela MainJFrame é aberta. A tela principal oferece botões para Cliente, PC, Alugar e Funcionário, cada um direcionando para uma janela de gerenciamento.

A estrutura do projeto no código-fonte está organizada nos pacotes dao, model, view, util, resources e images. O pacote model contém as entidades de domínio. O pacote dao contém as classes que executam SQL. O pacote view contém as interfaces Swing. O pacote util contém a ConnectionFactory, que cria conexões JDBC. O pacote resources contém o arquivo database.properties.

Pacote/arquivo	Responsabilidade observada no código
model	Entidades simples: Cliente, Funcionario, Computador e Aluguel.
dao	Operações de persistência: inserir, atualizar, excluir, buscar por ID, listar e contar.
view	Telas Java Swing: Login, menu principal, cliente, funcionário, computador e aluguel.
util	ConnectionFactory para carregar propriedades e abrir conexão MySQL.
resources/database.properties	URL, usuário e senha do banco de dados lanhousedb.
images	Recursos gráficos utilizados no projeto NetBeans/Swing.

3.1 Tecnologias utilizadas

Tecnologia	Uso no projeto
Java	Linguagem principal da aplicação.
Java Swing	Construção das telas desktop.
JDBC	Comunicação entre Java e MySQL.
MySQL	Banco relacional utilizado para persistência.
NetBeans/Ant	Estrutura do projeto, arquivos .form e build.xml.
DAO	Separação da camada de persistência.

3.2 Módulos do sistema

Módulo	Descrição fiel ao código
Login	Valida usuário e senha digitados comparando com valores fixos no código: admin e 123.
Principal	Apresenta botões de navegação para Cliente, PC, Alugar e Funcionário.
Clientes	Permite inserir, atualizar, excluir, listar e contar clientes.
Funcionários	Permite inserir, atualizar, excluir, listar e contar funcionários.
Computadores	Permite inserir, atualizar, excluir, listar e contar computadores, além de controlar status.
Aluguéis	Permite registrar, excluir, buscar/listar e contar aluguéis; o código contém intenção de verificar disponibilidade e alterar status do computador.

Mapa de Navegação das Interfaces

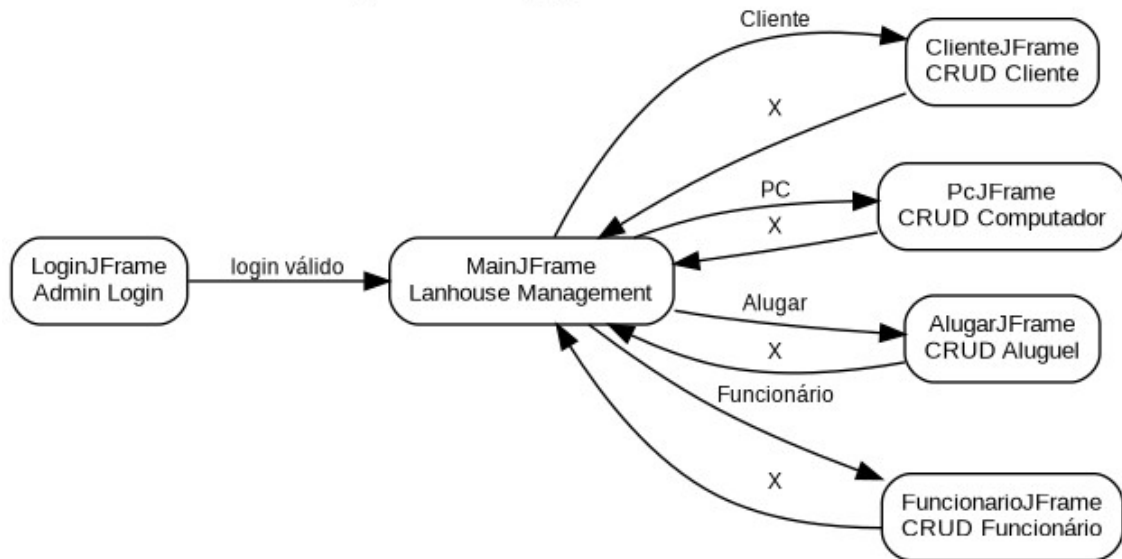


Figura 1 - Mapa de navegação das interfaces.

4 LEVANTAMENTO DE REQUISITOS

Os requisitos abaixo foram levantados a partir das classes JFrame, Model e DAO do projeto. Portanto, representam o comportamento implementado ou diretamente indicado pelo código-fonte, e não uma proposta abstrata independente da aplicação.

4.1 Requisitos funcionais

Código	Nome	Descrição
RF01	Realizar autenticação do administrador	O sistema deve permitir acesso ao menu principal mediante validação das credenciais na tela de login.
RF02	Exibir tela principal	Após login válido, o sistema deve abrir a tela Lanhouse Management com acesso aos módulos.
RF03	Cadastrar cliente	O sistema deve inserir cliente com id, nome, email e endereço.
RF04	Atualizar cliente	O sistema deve alterar nome, email e endereço de um cliente identificado por id.
RF05	Excluir cliente	O sistema deve excluir cliente pelo id informado.
RF06	Listar clientes	O sistema deve preencher a JTable com os clientes ordenados por id.
RF07	Contar clientes	O sistema deve retornar e exibir a quantidade de clientes cadastrados.
RF08	Cadastrar funcionário	O sistema deve inserir funcionário com idfuncionario, nomefuncionario, emailfuncionario e enderecofuncionario.
RF09	Atualizar funcionário	O sistema deve alterar dados de funcionário a partir do idfuncionario.
RF10	Excluir funcionário	O sistema deve excluir funcionário pelo idfuncionario.
RF11	Listar e contar funcionários	O sistema deve listar todos os funcionários e exibir a contagem total.
RF12	Cadastrar computador	O sistema deve inserir computador com id e status.
RF13	Atualizar status de computador	O sistema deve alterar o status de um computador cadastrado.
RF14	Excluir computador	O sistema deve excluir computador pelo id informado.
RF15	Listar e contar computadores	O sistema deve listar computadores ordenados por id e exibir a contagem total.
RF16	Registrar aluguel	O sistema deve inserir aluguel com idaluguel, nomecliente, idcomputador e qtdhoras.
RF17	Excluir aluguel	O sistema deve excluir registro de aluguel pelo idaluguel.
RF18	Listar e contar aluguéis	O sistema deve listar aluguéis e exibir total de registros.

RF19	Verificar disponibilidade do computador	O código contém método <code>estaDisponivel</code> em <code>ComputadorDAO</code> para consultar se o status é Disponível.
RF20	Alterar status do computador	O código contém método <code>alterarStatus</code> para atualizar status de computador, usado na intenção de marcar como Ocupado após aluguel.

4.2 Requisitos não funcionais

Código	Nome	Descrição
RNF01	Linguagem Java	O sistema deve ser desenvolvido em Java.
RNF02	Interface desktop	A interação deve ocorrer por telas Java Swing.
RNF03	Banco MySQL	A persistência deve utilizar MySQL, conforme <code>database.properties</code> e comandos SQL dos DAOs.
RNF04	Acesso por JDBC	O acesso ao banco deve utilizar <code>Connection</code> , <code>PreparedStatement</code> e <code>ResultSet</code> .
RNF05	Organização em camadas	O código deve separar View, Model, DAO e Util.
RNF06	Consultas parametrizadas	Os comandos SQL devem ser executados com <code>PreparedStatement</code> .
RNF07	Tratamento de exceções	As telas devem capturar exceções das operações e registrar no Logger ou exibir mensagens.
RNF08	Controle de estado do computador	O status deve estar restrito aos estados Disponível, Ocupado, Manutenção e Reservado conforme <code>JComboBox/estrutura</code> do sistema.
RNF09	Autenticação	A norma do trabalho pede login com usuário e senha criptografados; no código analisado, a autenticação está fixa em <code>admin/123</code> , sem criptografia.

4.3 Requisitos suplementares e regras de negócio

Código	Regra/observação	Descrição
RN01	Identificação manual	As telas exigem que o usuário informe o ID manualmente nos cadastros.
RN02	Cliente textual no aluguel	O aluguel armazena <code>nomeCliente</code> como texto, e não <code>idCliente</code> como chave estrangeira implementada no model.
RN03	Computador obrigatório no aluguel	O aluguel contém <code>idComputador</code> , que se relaciona ao computador utilizado.
RN04	Status do computador	A tela PC oferece estados

		Disponível, Ocupado, Manutenção e Reservado.
RN05	Login válido	Somente usuário admin e senha 123 abrem a tela principal na implementação atual.
RN06	Busca por ID nos DAOs	Os DAOs implementam métodos buscarPorId, usados para localizar registros pelo identificador.

5 CASOS DE USO

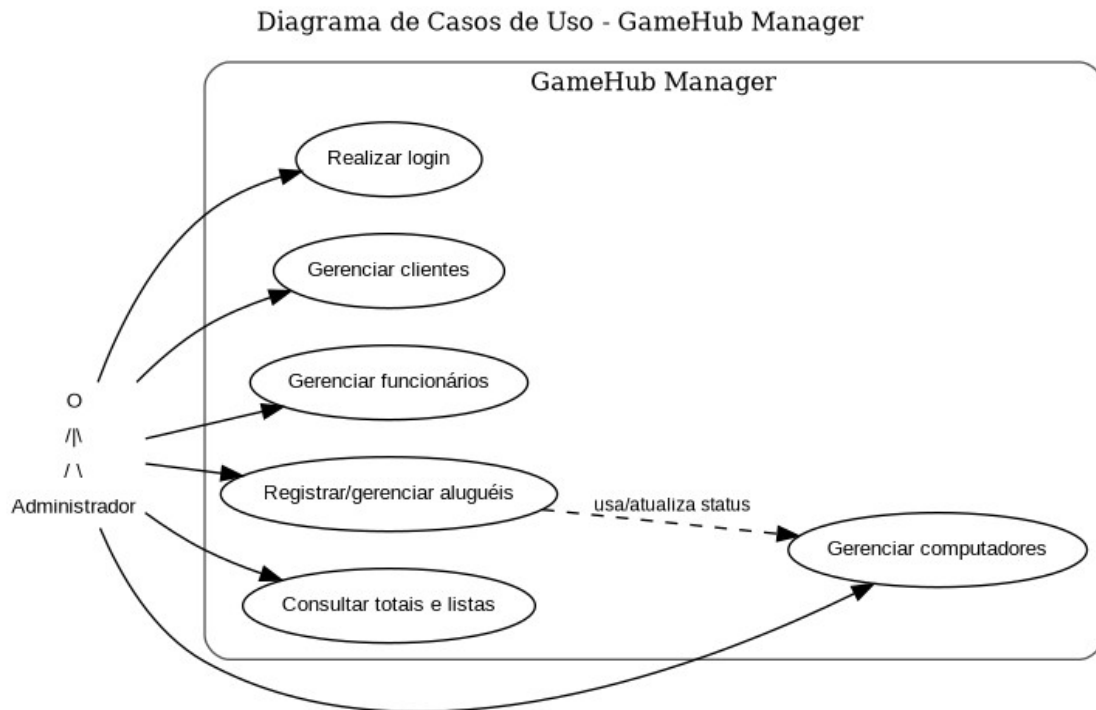


Figura 2 - Diagrama de Casos de Uso.

O ator principal identificado é o Administrador. O sistema não diferencia, no código atual, perfis de acesso como operador, gerente ou cliente final. Todas as telas administrativas são alcançadas a partir do menu principal após login válido.

UC01 - Realizar Login

Campo	Descrição
Objetivo	Permitir que o administrador acesse o menu principal do sistema.
Ator principal	Administrador
Pré-condições	A aplicação deve estar iniciada na tela LoginJFrame.
Fluxo principal	1. O administrador informa username. 2. O administrador informa password. 3. O administrador clica em Login. 4. A tela compara os valores digitados com admin e 123. 5. Se os dados forem válidos, LoginJFrame é ocultada e MainJFrame é aberta.
Cenários alternativos	Não há fluxo alternativo implementado para recuperação de senha ou cadastro de usuário.
Exceções	Se usuário ou senha forem inválidos, o sistema exibe mensagem de erro por JOptionPane.
Pós-condições	Menu principal aberto para uso.
Regras de negócio	Na implementação atual, as credenciais são fixas no código e não são criptografadas.

UC02 - Gerenciar Clientes

Campo	Descrição
Objetivo	Cadastrar, atualizar, excluir, listar e contar clientes.

Ator principal	Administrador
Pré-condições	Administrador autenticado e tela ClienteJFrame aberta.
Fluxo principal	1. O administrador informa ID, nome, email e endereço. 2. Ao clicar em Insert, a View cria um objeto Cliente. 3. ClienteDAO executa INSERT na tabela cliente. 4. Ao clicar em Update, ClienteDAO executa UPDATE no registro informado. 5. Ao clicar em Delete, ClienteDAO exclui pelo ID. 6. A tabela é recarregada com listarTodos. 7. O botão Total de Clientes consulta COUNT no banco.
Cenários alternativos	A operação de atualização reaproveita os campos da mesma tela. A exclusão utiliza o ID informado.
Exceções	Campos numéricos inválidos podem gerar NumberFormatException; falhas SQL são registradas no Logger. O código contém método de validação de e-mail, mas a documentação registra apenas o que aparece na tela e DAO.
Pós-condições	Cliente persistido, alterado, excluído ou listado.
Regras de negócio	Cliente possui id, nome, email e endereço.

UC03 - Gerenciar Funcionários

Campo	Descrição
Objetivo	Administrar registros de funcionários da lan house.
Ator principal	Administrador
Pré-condições	Administrador autenticado e tela FuncionarioJFrame aberta.
Fluxo principal	1. O administrador informa ID, nome, email e endereço do funcionário. 2. A tela cria um objeto Funcionario. 3. FuncionarioDAO executa INSERT, UPDATE ou DELETE conforme botão acionado. 4. A tabela de funcionários é recarregada após operações. 5. O botão Total de Funcionário exibe COUNT da tabela funcionario.
Cenários alternativos	Não há perfis de permissão diferentes implementados para funcionário.
Exceções	Erros de conexão, SQL ou conversão são tratados por exceção/log.
Pós-condições	Funcionário persistido, alterado, removido ou listado.
Regras de negócio	Tabela funcionario usa idfuncionario, nomefuncionario, emailfuncionario e enderecofuncionario.

UC04 - Gerenciar Computadores

Campo	Descrição
Objetivo	Controlar computadores cadastrados e seus status.
Ator principal	Administrador
Pré-condições	Administrador autenticado e tela PcJFrame aberta.
Fluxo principal	1. O administrador informa ID do computador. 2. O administrador escolhe ou informa status do computador.

	3. A tela cria objeto Computador. 4. ComputadorDAO persiste, atualiza ou remove o computador. 5. A tabela de computadores é atualizada. 6. O botão Total PC exibe o total de computadores.
Cenários alternativos	A alteração de status pode representar disponibilidade, ocupação, reserva ou manutenção.
Exceções	IDs inválidos ou falha de banco podem impedir a operação.
Pós-condições	Computador cadastrado ou atualizado com status.
Regras de negócio	Status previstos: Disponível, Ocupado, Manutenção e Reservado.

UC05 - Gerenciar Aluguéis

Campo	Descrição
Objetivo	Registrar e consultar aluguéis realizados na lan house.
Ator principal	Administrador
Pré-condições	Administrador autenticado, tela AlugarJFrame aberta e computador cadastrado.
Fluxo principal	1. O administrador informa ID do aluguel, nome do cliente, ID do computador e quantidade de horas. 2. A tela cria objeto Aluguel. 3. AluguelDAO executa INSERT na tabela aluguel. 4. A tabela de aluguéis é recarregada. 5. O total de aluguéis pode ser consultado.
Cenários alternativos	Exclusão de aluguel pelo ID informado. A versão pretendida do DAO indica verificação de disponibilidade do computador e atualização para Ocupado.
Exceções	Computador indisponível, falha SQL ou conversão inválida podem impedir registro.
Pós-condições	Aluguel registrado e associado ao ID do computador.
Regras de negócio	Aluguel contém nomeCliente textual e idComputador numérico.

6 DIAGRAMA DE ATIVIDADES

O fluxo escolhido para o diagrama de atividades é o registro de aluguel, por ser o processo que integra cliente, computador, aluguel e alteração de disponibilidade do equipamento. A documentação considera o fluxo implementado na tela e nos DAOs, registrando também a intenção de controle de disponibilidade presente no código.

Diagrama de Atividades - Registro de Aluguel

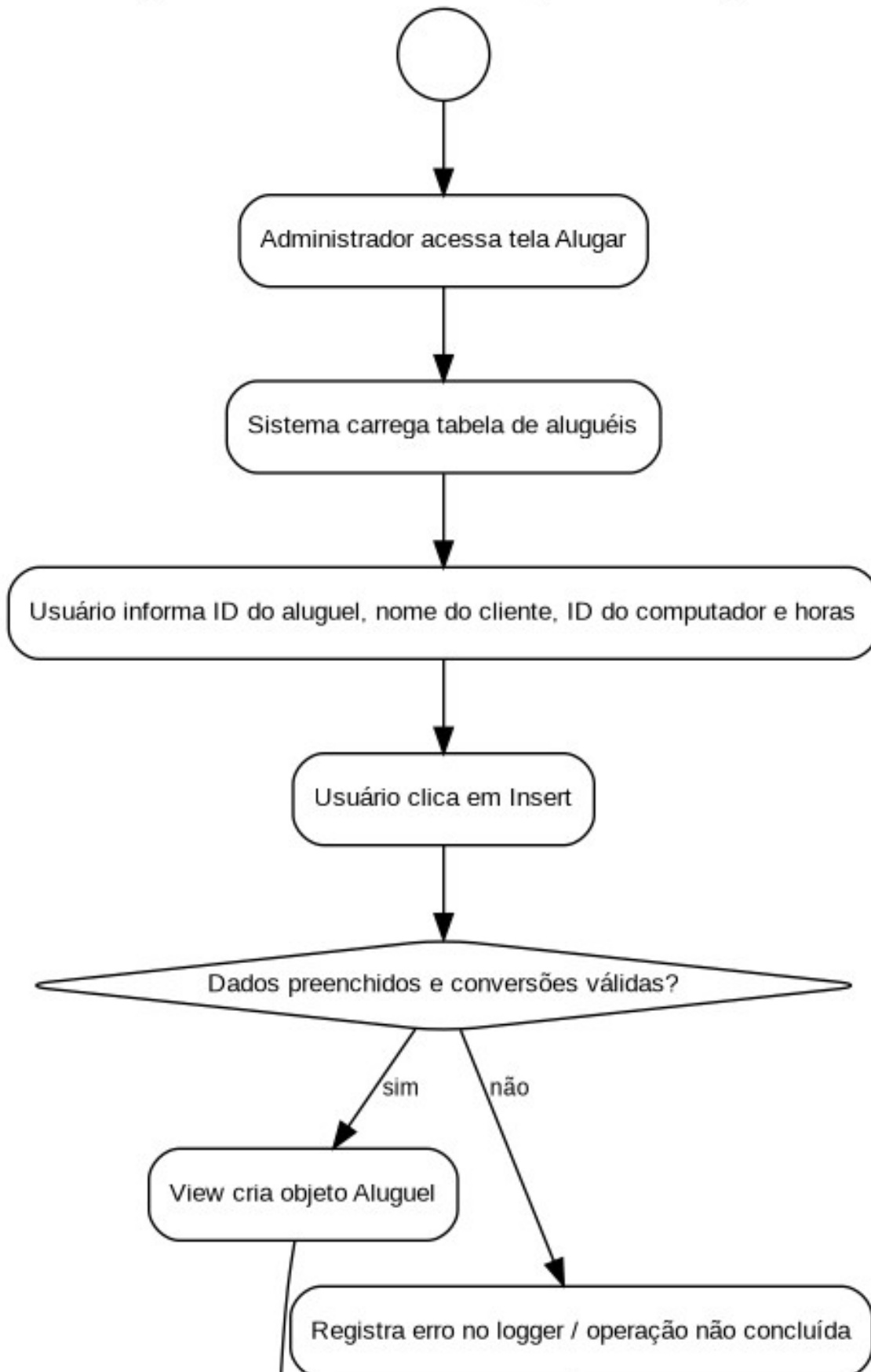


Figura 3 - Diagrama de Atividades do registro de aluguel.

O processo começa com o administrador acessando a tela Alugar. O sistema lista os alugueis existentes. Em seguida, o administrador informa os dados do aluguel e confirma a inserção. A View monta um objeto Aluguel e chama o DAO. O DAO executa SQL no banco. Em caso de dados inválidos ou falha de banco, a operação é interrompida e o erro é tratado pelo mecanismo de exceção/log.

7 DIAGRAMA DE MÁQUINA DE ESTADOS

A entidade mais adequada para máquina de estados é Computador, pois o sistema trabalha diretamente com o atributo status. A tela PC apresenta estados de disponibilidade e o DAO possui métodos para consultar e alterar o status de um computador.

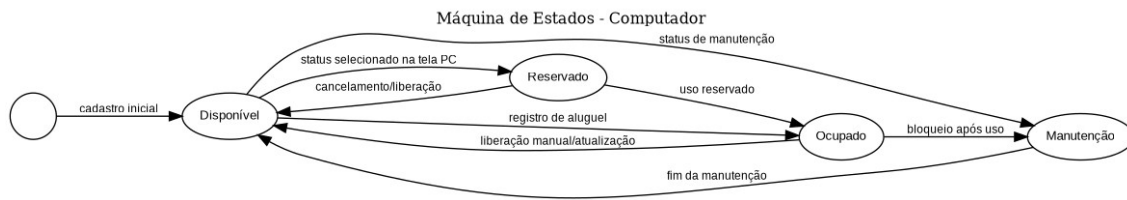


Figura 4 - Máquina de Estados da entidade Computador.

Os estados documentados são Disponível, Ocupado, Reservado e Manutenção. O estado Disponível representa equipamento apto para aluguel. Ocupado representa equipamento em uso. Reservado representa bloqueio para uso futuro. Manutenção representa indisponibilidade técnica. As transições podem ocorrer por atualização manual na tela PC ou por operação de aluguel, quando a intenção do código é alterar o computador para Ocupado.

8 DIAGRAMAS DE SEQUÊNCIA

8.1 Login

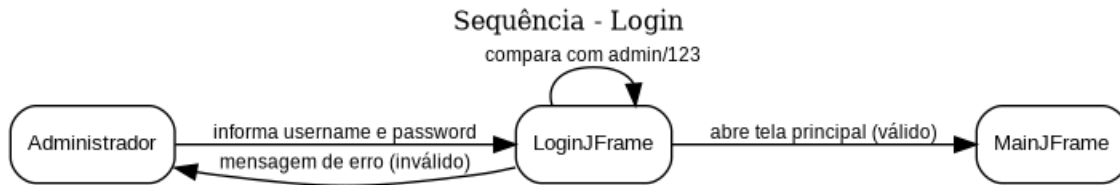


Figura 5 - Diagrama de Sequência do Login.

A sequência de login é simples e ocorre sem consulta ao banco. LoginJFrame lê os campos username e password, compara os valores com as credenciais fixas admin e 123 e, em caso positivo, instancia MainJFrame. Em caso negativo, apresenta mensagem de usuário ou senha inválidos.

8.2 CRUD de Cliente

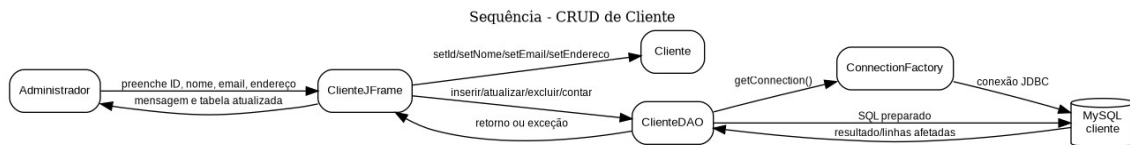


Figura 6 - Diagrama de Sequência do CRUD de Cliente.

O fluxo de cliente representa o padrão repetido em ClienteJFrame, FuncionarioJFrame e PcJFrame. A tela captura dados dos campos, monta um objeto de modelo, chama o DAO correspondente, o DAO solicita conexão à ConnectionFactory, executa SQL com PreparedStatement e retorna para a tela, que mostra mensagem e recarrega a JTable.

8.3 Registro de Aluguel

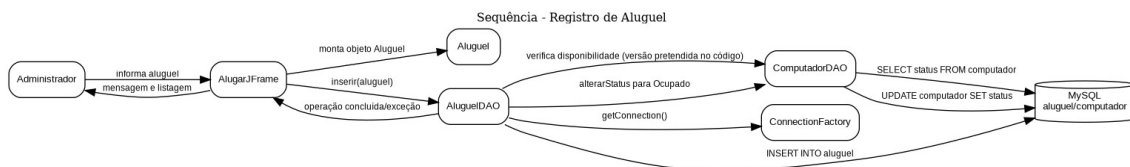


Figura 7 - Diagrama de Sequência do Registro de Aluguel.

O aluguel é o caso em que o código indica dependência direta com ComputadorDAO. A versão pretendida de inserir aluguel verifica se o computador está disponível e altera o status para Ocupado após o INSERT. Essa intenção é coerente com a regra de negócio de uma lan house, na qual um computador alugado não deve continuar disponível para outro uso simultâneo.

9 MODELO CONCEITUAL

O modelo conceitual foi extraído das classes de domínio e dos relacionamentos implícitos nas telas e DAOs. As entidades do domínio são Cliente, Funcionário, Computador e Aluguel. O Funcionário aparece como entidade administrável, não como ator autenticado persistido. O ator operacional do sistema é representado como Administrador nos casos de uso.

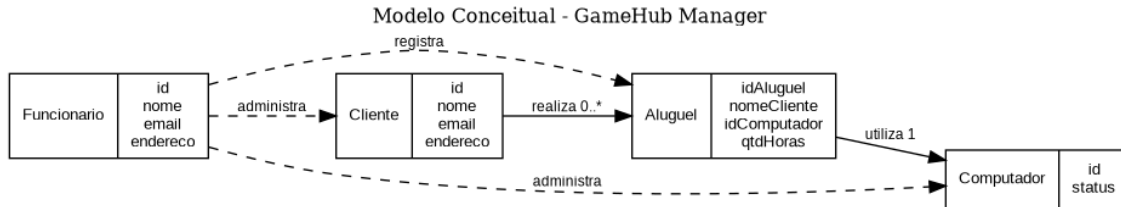


Figura 8 - Modelo Conceitual.

Cliente possui identificação, nome, e-mail e endereço. Computador possui identificação e status. Aluguel possui identificação, nome do cliente, id do computador e quantidade de horas. Funcionário possui identificação, nome, e-mail e endereço. O relacionamento mais forte do banco é entre Aluguel e Computador por idComputador. A ligação com Cliente é textual pelo campo nomeCliente, não por chave idCliente no objeto Aluguel.

9.1 Entidades e atributos

Entidade	Atributos	Origem no código
Cliente	id, nome, email, endereco	model.Cliente e ClienteDAO
Funcionario	id, nome, email, endereco	model.Funcionario e FuncionarioDAO
Computador	id, status	model.Computador e ComputadorDAO
Aluguel	idAluguel, nomeCliente, idComputador, qtdHoras	model.Aluguel e AluguelDAO

10 MODELO RELACIONAL

O modelo relacional foi inferido diretamente das strings SQL dos DAOs. Os nomes das tabelas e colunas abaixo refletem os comandos INSERT, UPDATE, DELETE, SELECT e COUNT existentes no código.

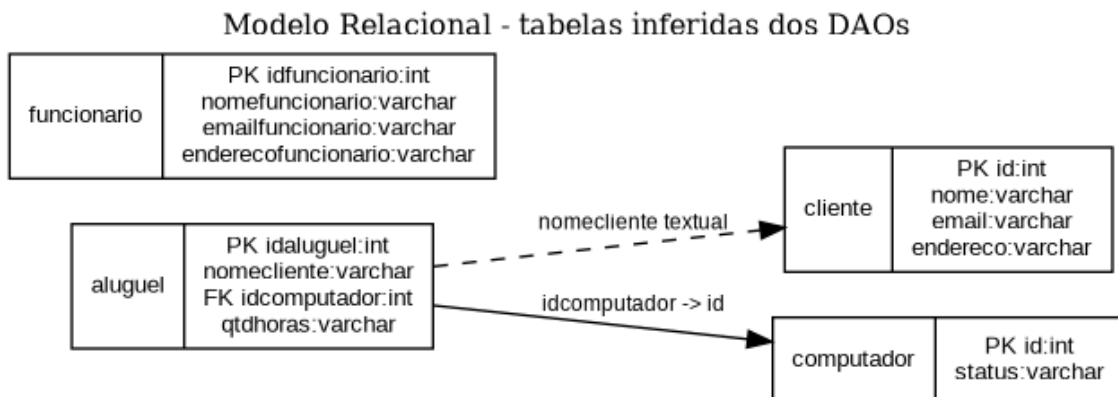


Figura 9 - Modelo Relacional inferido dos DAOs.

Tabela	Colunas identificadas	Chave/relacionamento
--------	-----------------------	----------------------

cliente	id, nome, email, endereco	id como chave primária lógica.
funcionario	idfuncionario, nomefuncionario, emailfuncionario, enderecofuncionario	idfuncionario como chave primária lógica.
computador	id, status	id como chave primária lógica.
aluguel	idaluguel, nomecliente, idcomputador, qtdhoras	idaluguel como chave primária lógica; idcomputador referencia computador.id.

10.1 DDL sugerido coerente com o código

O código não traz arquivo SQL de criação das tabelas. A estrutura abaixo é uma sugestão fiel aos nomes usados pelos DAOs, útil para recriar o banco lanhousedb:

```
CREATE TABLE cliente (
  id INT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL,
  endereco VARCHAR(180)
);

CREATE TABLE funcionario (
  idfuncionario INT PRIMARY KEY,
  nomefuncionario VARCHAR(100) NOT NULL,
  emailfuncionario VARCHAR(120) NOT NULL,
  enderecofuncionario VARCHAR(180)
);

CREATE TABLE computador (
  id INT PRIMARY KEY,
  status VARCHAR(30) NOT NULL
);

CREATE TABLE aluguel (
  idaluguel INT PRIMARY KEY,
  nomecliente VARCHAR(100) NOT NULL,
  idcomputador INT NOT NULL,
  qtdhoras VARCHAR(20) NOT NULL,
  FOREIGN KEY (idcomputador) REFERENCES computador(id)
);
```

11 DIAGRAMA DE CLASSES

O diagrama de classes mostra a separação entre entidades de domínio, DAOs, telas e utilitário de conexão. As entidades são classes simples, com atributos privados e métodos de acesso. Os DAOs dependem da ConnectionFactory para obter conexão com o banco. As Views instanciam DAOs e modelos para realizar as operações.

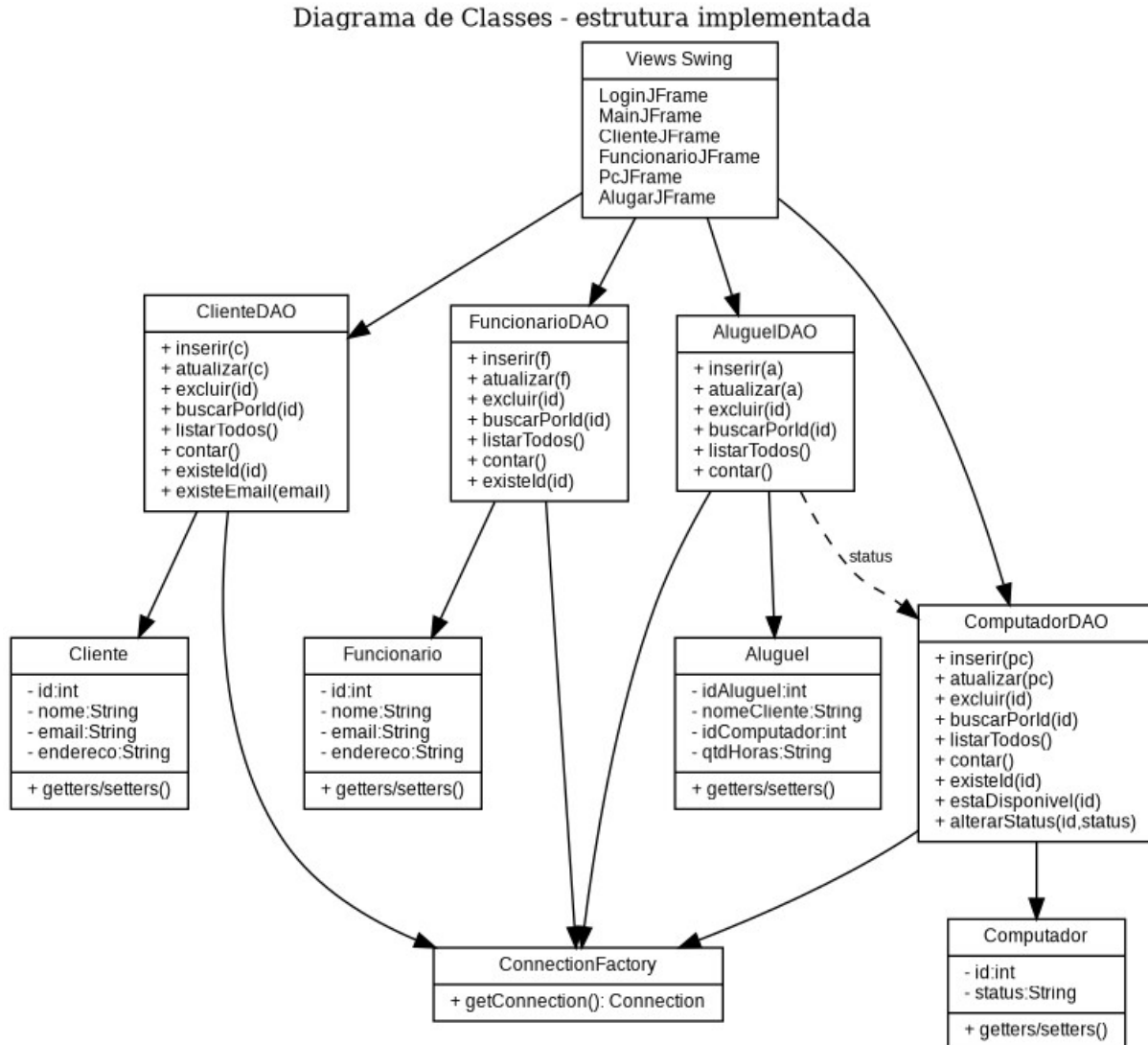


Figura 10 - Diagrama de Classes do GameHub Manager.

11.1 Descrição das classes Model

Classe	Atributos	Descrição
Cliente	id:int; nome:String; email:String; endereco:String	Representa cliente cadastrado na lan house.
Funcionario	id:int; nome:String; email:String; endereco:String	Representa funcionário cadastrado no sistema.
Computador	id:int; status:String	Representa computador disponível, ocupado, reservado ou em manutenção.
Aluguel	idAluguel:int; nomeCliente:String; idComputador:int; qtdHoras:String	Representa registro de uso/aluguel de um computador por determinado cliente.

11.2 Descrição das classes DAO

DAO	Métodos principais	Tabela acessada
ClienteDAO	inserir, atualizar, excluir, buscarPorId, listarTodos, contar, existeld, existeEmail	cliente
FuncionarioDAO	inserir, atualizar, excluir, buscarPorId, listarTodos, contar, existeld	funcionario
ComputadorDAO	inserir, atualizar, excluir, buscarPorId, listarTodos, contar, existeld, estaDisponivel, alterarStatus	computador
AluguelDAO	inserir, atualizar, excluir, buscarPorId, listarTodos, contar	aluguel

12 ARQUITETURA DO SISTEMA

A arquitetura do GameHub Manager segue uma divisão em camadas simples, comum em aplicações Java desktop acadêmicas. A camada View concentra os JFrames e interage diretamente com o usuário. A camada Model representa os dados do domínio. A camada DAO executa comandos SQL. A camada Util centraliza a criação de conexões.

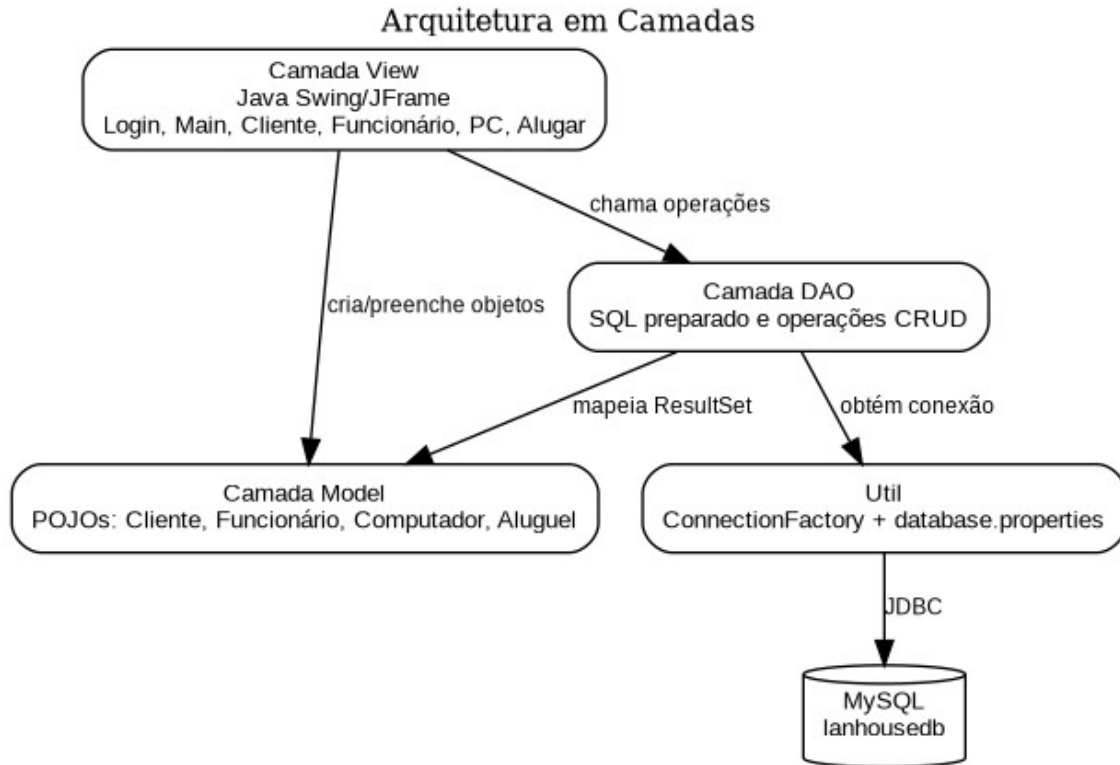


Figura 11 - Arquitetura em camadas.

12.1 Camada View

A camada View contém LoginJFrame, MainJFrame, ClienteJFrame, FuncionarioJFrame, PcJFrame e AlugarJFrame. Essas classes possuem componentes gráficos Swing, métodos initComponents gerados pelo NetBeans e métodos actionPerformed que respondem aos eventos dos botões. A View também instancia objetos Model e DAOs, o que torna a arquitetura simples, porém com acoplamento direto entre interface e persistência.

12.2 Camada Model

A camada Model contém POJOs sem lógica complexa. O papel dessas classes é transportar dados entre a interface e o DAO. Cada entidade possui construtor vazio, construtor com parâmetros e métodos getters/setters.

12.3 Camada DAO

A camada DAO contém a lógica de persistência. Os comandos SQL são escritos como strings e executados por PreparedStatement. As consultas de listagem percorrem ResultSet e montam listas de objetos. A contagem de registros é feita por SELECT COUNT(*).

12.4 Camada Util e configuração

ConnectionFactory lê o arquivo resources/database.properties e abre conexão com o MySQL. O arquivo configura URL jdbc:mysql://localhost:3306/lanhousedb, usuário root e senha local. Por segurança, em uma versão de produção a senha não deveria ficar salva no repositório.

13 DESCRIÇÃO DAS INTERFACES

13.1 LoginJFrame

Tela inicial do sistema, intitulada Admin Login. Contém campos Username e Password e botão Login. Ao acionar o botão, a tela lê os textos digitados, compara com admin e 123 e abre MainJFrame se a validação for positiva. Se a validação falhar, exibe JOptionPane com a mensagem Usuário ou senha inválido.

13.2 MainJFrame

Tela principal intitulada Lanhouse Management. Contém botões Cliente, PC, Alugar, Funcionário e X. Cada botão abre a respectiva janela de gerenciamento. O botão X encerra/fecha a navegação principal conforme tratamento implementado.

13.3 ClienteJFrame

Tela de gerenciamento de clientes. Possui tabela com colunas ID, Nome, Email e Endereço; campos para preenchimento dos mesmos dados; botões Insert, Update, Delete e Total de Clientes. A tela chama ClienteDAO para persistir dados e recarrega a tabela por listarTodos.

13.4 FuncionarioJFrame

Tela de gerenciamento de funcionários. A estrutura é semelhante à tela de clientes, com campos ID, Nome, Email e Endereço, tabela de listagem e botões Insert, Update, Delete e Total de Funcionário. As operações são encaminhadas ao FuncionarioDAO.

13.5 PcJFrame

Tela de gerenciamento de computadores. Possui campo ID, controle de status e tabela com ID e Status. O código apresenta status como Disponível, Ocupado, Manutenção e Reservado em uma versão da tela. Os botões Insert, Update, Delete e Total PC acionam ComputadorDAO.

13.6 AlugarJFrame

Tela de gerenciamento de aluguéis. Trabalha com ID do aluguel, nome do cliente, ID do computador e quantidade de horas. A tela lista aluguéis em JTable e executa operações de inserção, exclusão, listagem e contagem por meio de AluguelDAO. A documentação registra a associação do aluguel ao computador por idComputador.

14 OBSERVAÇÕES TÉCNICAS DO CÓDIGO-FONTE

Durante a análise do projeto foram identificados pontos relevantes que precisam constar na documentação por fidelidade ao código. Essas observações não invalidam a proposta do sistema, mas indicam melhorias necessárias para uma entrega mais robusta.

Ponto observado	Impacto técnico	Recomendação
Login fixo em admin/123	Não atende plenamente ao requisito de usuário e senha criptografados; não consulta banco.	Criar tabela de usuários/administradores, armazenar hash de senha e validar por DAO.
Senha do banco em	Credencial local fica visível no	Usar variáveis de ambiente ou

database.properties	projeto.	arquivo fora do repositório em produção.
ID informado manualmente	Pode gerar colisões e erros se o usuário repetir ID.	Usar AUTO_INCREMENT no MySQL e recuperar chave gerada.
Aluguel usa nomeCliente textual	Dificulta integridade referencial com cliente.	Trocar por idCliente como chave estrangeira.
Arquivos com blocos duplicados em alguns DAOs/Views	Pode causar erro de compilação se houver duas declarações de classe no mesmo arquivo ou trechos antigos concatenados.	Manter apenas uma versão final de cada classe.
qtdHoras como String	Quantidade de horas é dado numérico, mas está como texto.	Usar int ou decimal conforme regra de cobrança.
Tratamento de erro majoritariamente por Logger	Usuário pode não receber feedback claro em todos os erros.	Exibir mensagens amigáveis na interface e registrar detalhes técnicos no log.

Esses pontos foram documentados porque a norma do trabalho valoriza a documentação do software e a coerência entre implementação e projeto. Assim, a documentação não afirma que há criptografia ou autenticação por banco, pois isso não aparece no código analisado.

15 CONCLUSÃO

O GameHub Manager cumpre a função de sistema desktop para gerenciamento básico de uma lan house. A aplicação reúne telas para login, navegação principal, cadastro de clientes, funcionários, computadores e registros de aluguel. A organização em pacotes model, dao, view e util demonstra aplicação de conceitos de Programação Orientada a Objetos, padrão DAO, JDBC e banco de dados relacional.

A documentação apresentada relaciona os elementos do código-fonte com requisitos, casos de uso e diagramas UML. O sistema possui uma base clara para evolução: o controle de computadores por status, a persistência em MySQL e a divisão por entidades permitem ampliar funcionalidades como relatórios, autenticação por banco, cálculo de valores de aluguel, controle de tempo, histórico de uso e melhor integridade entre cliente e aluguel.

Como melhoria principal, recomenda-se adequar o login ao requisito de senha criptografada, remover duplicidades de arquivos, transformar identificadores em auto incremento, substituir nomeCliente por referência a Cliente e melhorar o tratamento de erros na interface. Ainda assim, a estrutura atual é suficiente para demonstrar os principais conceitos exigidos em um projeto acadêmico de Java com banco de dados e documentação UML.

16 REFERÊNCIAS

- WAZLAWICK, Raul Sidnei. Análise e Projeto de Sistemas de Informação Orientados a Objetos.
- BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: Guia do Usuário.
- ORACLE. Java Platform, Standard Edition Documentation.
- ORACLE. Java Swing Tutorial and API Documentation.
- MYSQL. MySQL Reference Manual.
- GAMMA, Erich et al. Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos.